Proyecto final — ZONA FITNESS

Plataforma web promocional, comercial y de soporte

Plataforma web promocional, comercial y de soporte

Proyecto: Zona Fitness **Tecnologías:** Angular 22, TypeScript, Node.js, Express y PostgreSQL 18 **Estado de entrega:** Finalizado y validado **Fecha de cierre:** Octubre de 2026

1. Resumen ejecutivo

Zona Fitness es una plataforma web responsive que centraliza la presencia digital del gimnasio y facilita el recorrido del visitante desde el descubrimiento de los servicios hasta la consulta de planes, productos, horarios y condiciones de inscripción. La solución también incluye un espacio autenticado para soporte, gestión de usuarios, carrito persistido, checkout y administración de la información operativa.

El proyecto fue construido con una separación clara entre:

- **Experiencia pública:** información del gimnasio, servicios, planes, horarios, espacios, productos, redes y política de datos.
- **Experiencia autenticada:** cuenta del usuario, pedidos, membresías, tickets y soporte.
- **Administración:** gestión de usuarios, roles, auditoría, inventario y datos operativos.

Al cierre del proyecto se considera alcanzado el alcance aprobado: la aplicación compila, dispone de rutas funcionales, aplica controles de acceso, conserva los datos en PostgreSQL y cuenta con pruebas automatizadas y documentación de mantenimiento.

2. Objetivos y metas alcanzadas

Objetivo general

Diseñar e implementar una plataforma digital confiable, accesible y responsive para presentar la oferta de Zona Fitness, apoyar la conversión de visitantes y organizar las operaciones básicas de usuarios, pedidos y soporte.

Metas verificadas

Meta	Resultado final	Evidencia
Presentar la identidad visual del gimnasio	Alcanzada	Logo, icono, paleta de marca, tipografía, componentes y footer
Crear una navegación clara	Alcanzada	Rutas para inicio, servicios, planes, horarios, espacio, productos y privacidad
Publicar servicios y planes	Alcanzada	Secciones informativas y páginas dedicadas reutilizables
Incorporar horarios, ubicación y galería	Alcanzada	Módulos de horarios, espacio y ubicación
Ofrecer productos y recorrido de compra	Alcanzada	Catálogo, carrito persistido, cálculo en servidor y checkout
Proteger el acceso de usuarios	Alcanzada	JWT, bcrypt, guards de autenticación y RBAC
Cumplir la regla de edad y consentimiento	Alcanzada	Validación de edad, acudiente para menores y registros de consentimiento
Proteger los datos personales	Alcanzada	Política de datos, consentimiento versionado y auditoría
Permitir soporte interno	Alcanzada	Registro, login, tickets y gestión administrativa
Garantizar adaptación a dispositivos	Alcanzada	Layout responsive, menú móvil y ajustes específicos para móvil
Validar calidad técnica	Alcanzada	Pruebas unitarias, pruebas API, pruebas de base de datos y build
Documentar el mantenimiento	Alcanzada	Manual de cliente, guía técnica, arquitectura, flujo y este documento

3. Responsabilidades de los miembros

La siguiente distribución permite explicar el trabajo del equipo y evitar que las responsabilidades queden concentradas en una sola persona. Los nombres pueden reemplazarse por los integrantes reales del grupo al momento de la sustentación.

Rol / miembro	Responsabilidades principales	Entregables asociados
Líder de proyecto / Product Owner	Definir el alcance, priorizar requisitos, validar los objetivos, coordinar reuniones y aprobar la entrega final	Alcance, cronograma, matriz de metas y acta de cierre
Analista de requisitos y documentación	Levantar necesidades, definir historias de usuario, documentar reglas de negocio y consolidar la memoria del proyecto	Requisitos, manual, guía técnica y documento final
Diseñador UX/UI	Definir arquitectura de información, wireframes, jerarquía visual, accesibilidad, responsive y aplicación de la marca	Wireframe, propuesta visual, prototipo y sistema de estilos
Desarrollador frontend	Construir componentes Angular, rutas, navegación, formularios, estado visual, carrusel, carrito y vistas responsive	`src/app/`, estilos globales, componentes y pruebas Angular
Desarrollador backend	Implementar API REST, autenticación, autorización, validaciones, pedidos, tickets y manejo de errores	`server/src/`, endpoints, middleware y pruebas API

Rol / miembro	Responsabilidades principales	Entregables asociados
Administrador de base de datos	Modelar entidades, relaciones, restricciones, migraciones, índices, respaldos y datos iniciales	Migraciones SQL, esquema relacional, scripts de backup y guía de pgAdmin
Responsable de QA y seguridad	Diseñar casos de prueba, verificar accesibilidad, revisar IDOR/RBAC, validar reglas de edad y ejecutar regresiones	Suite automatizada, matriz de verificación y reporte de calidad
Responsable de despliegue y entrega	Preparar build, variables de entorno, publicación, evidencias, control de versiones y presentación	Build de producción, checklist de entrega y demostración

Coordinación y control

Cada integrante es responsable de entregar cambios revisables, documentar decisiones importantes y reportar bloqueos. El líder integra los resultados, pero no sustituye la responsabilidad técnica de cada rol. Las decisiones que afectan datos personales, pagos, roles o reglas de edad deben revisarse conjuntamente por desarrollo, base de datos y QA.

4. Arquitectura general de la solución

```
flowchart LR
    U[Visitante o usuario] --> B[Navegador]
    B --> F[Angular 22<br/>Frontend responsive]
    F --> R[Router y guards]
    F --> A[API REST]
    A --> E[Node.js + Express]
    E --> S[Servicios de autenticación,<br/>catálogo, pedidos y soporte]
    S --> D[(PostgreSQL 18)]
    E --> L[Logs y auditoría]
    D --> P[Políticas, migraciones<br/>y datos persistidos]
```

Capas

1. **Presentación:** componentes standalone de Angular, HTML semántico y CSS responsive.
2. **Navegación:** rutas lazy-loaded, `authGuard` y `roleGuard`.
3. **Aplicación:** servicios de autenticación, carrito, pedidos, membresías, tickets y validaciones.
4. **API:** endpoints Express con autenticación JWT, validación de entrada y respuestas HTTP consistentes.
5. **Persistencia:** PostgreSQL 18 con claves foráneas, `CHECK`, índices, migraciones y bitácora de auditoría.

5. Estructura DOM de la aplicación

La estructura siguiente representa el DOM lógico de la experiencia principal. Los componentes Angular generan este árbol en tiempo de ejecución y conservan regiones semánticas para navegación y accesibilidad.

```
html
■■■ body
```

```
■■■ app-root
    ■■■ a.skip-link
    ■   ■■■ "Saltar al contenido"
    ■■■ header.site-header
    ■   ■■■ div.header-main-bar
    ■   ■   ■■■ a.brand
    ■   ■   ■   ■■■ img.brand-mark
    ■   ■   ■   ■■■ span.brand-wordmark
    ■   ■   ■■■ nav.primary-nav[aria-label="Navegación principal"]
    ■   ■   ■   ■■■ a → /servicios
    ■   ■   ■   ■■■ a → /planes
    ■   ■   ■   ■■■ a → /horarios
    ■   ■   ■   ■■■ a → /espacio
    ■   ■   ■   ■■■ a → /productos
    ■   ■   ■■■ div.header-controls
    ■   ■       ■■■ a.cart-nav-link → /carrito
    ■   ■       ■■■ div.session (si existe sesión)
    ■   ■       ■   ■■■ a → /mi-cuenta o /users
    ■   ■       ■   ■■■ a → /tickets
    ■   ■       ■   ■■■ button.user-profile-trigger
    ■   ■       ■■■ button.mobile-menu-trigger
    ■   ■■■ nav.mobile-nav (cuando el menú está abierto)
    ■■■ main#main-content
    ■   ■■■ router-outlet
    ■       ■■■ home-page
    ■           ■■■ section.hero[aria-labelledby="hero-title"]
    ■           ■   ■■■ div.hero-copy
    ■           ■   ■   ■■■ p.eyebrow
    ■           ■   ■   ■■■ h1#hero-title
    ■           ■   ■   ■■■ p.hero-description
    ■           ■   ■   ■■■ div.hero-actions
    ■           ■   ■■■ div.hero-visual[role="region"]
    ■           ■       ■■■ img.hero-slide-image
    ■           ■       ■■■ div.hero-caption[aria-live="polite"]
    ■           ■       ■■■ div.hero-progress
    ■           ■       ■■■ div.hero-slider-controls
    ■           ■■■ section#servicios.intro
    ■           ■   ■■■ div.service-grid
    ■           ■       ■■■ article.service-card × 4
    ■           ■■■ section#planes.plans-section
    ■           ■   ■■■ div.plan-grid
    ■           ■       ■■■ article.plan-card × 4
    ■           ■■■ section#horarios.hours-section
    ■           ■■■ section#espacio.space-section
    ■           ■■■ section#productos.products-section
    ■           ■■■ section#inscripcion.registration-section
    ■■■ app-site-footer
        ■■■ footer.site-footer
            ■■■ div.footer-main
            ■   ■■■ div.footer-branding
            ■   ■■■ nav.footer-navigation
            ■   ■■■ div.footer-contact
            ■■■ div.footer-bottom
```

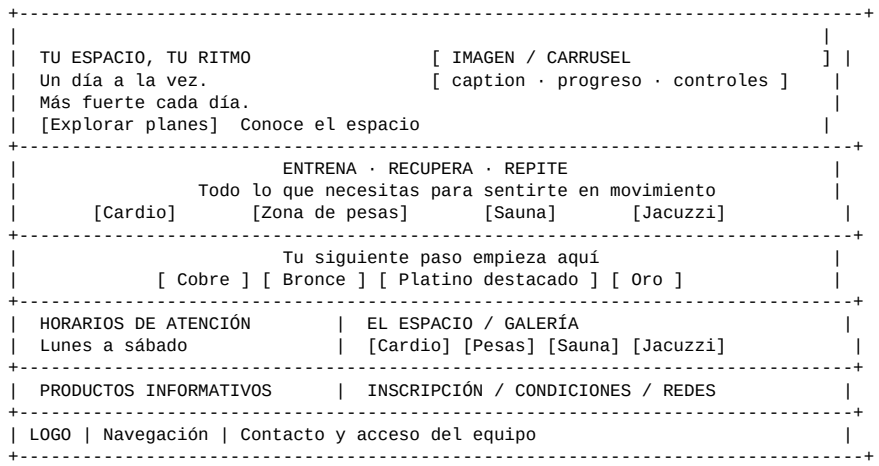
Criterios DOM y accesibilidad

- Un único `h1` identifica la vista principal.
- Las secciones usan `aria-labelledby` y títulos jerárquicos.
- Los controles interactivos son botones o enlaces nativos.
- El carrusel anuncia el contenido mediante `aria-live` y ofrece pausa manual.
- Existe enlace para saltar directamente al contenido.
- Las imágenes informativas tienen texto alternativo; las decorativas usan `alt=""`.
- La navegación móvil mantiene el orden de foco y no depende únicamente del color.

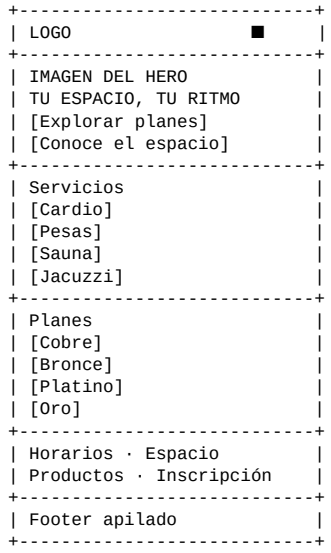
6. Wireframe funcional

6.1 Vista de escritorio

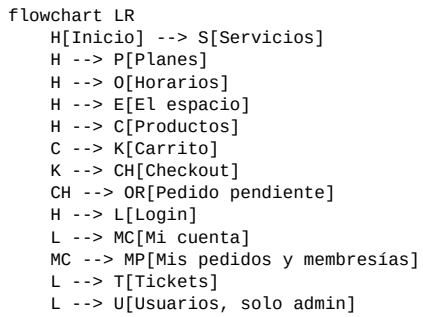
```
+-------------------------------------------------------------------------------+
| LOGO ZONA FITNESS | Servicios | Planes | Horarios | Espacio | Productos | ■  |
```



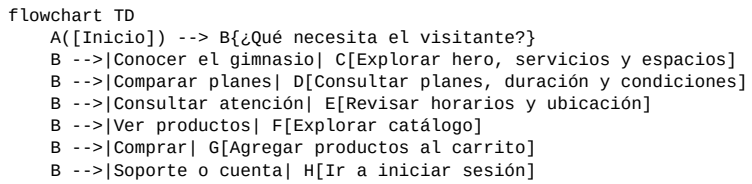
6.2 Vista móvil



6.3 Recorridos principales



7. Diagrama de flujo del sistema



```
    C --> I{¿Desea continuar?}
    D --> I
    E --> I
    I -->|Sí| J[Consultar inscripción y reglas de edad]
    I -->|No| Z([Fin de visita])
    J --> Z

    F --> G
    G --> K[Carrito persistido]
    K --> L{¿Usuario autenticado?}
    L -->|No| M[Solicitar login o registro]
    M --> N{¿Credenciales válidas?}
    N -->|No| M
    N -->|Sí| O[Checkout]
    L -->|Sí| O
    O --> P{¿Productos con precio confirmado?}
    P -->|No| Q[Informar y bloquear pedido]
    P -->|Sí| R[Calcular totales en servidor]
    R --> S[Crear pedido y vaciar carrito]
    S --> T[Mostrar código ZF-ORD]
    Q --> Z
    T --> Z

    H --> N
    N -->|Sí| V{¿Rol administrador?}
    V -->|Sí| W[Usuarios, tickets, auditoría e inventario]
    V -->|No| X[Cuenta, pedidos, membresías y tickets]
    W --> Z
    X --> Z
```

8. Modelo de datos y reglas de negocio

El modelo relacional contempla usuarios, roles, perfiles, socios, planes, membresías, pagos, asistencia, productos, movimientos de inventario, tickets y auditoría.

```
erDiagram
    ROLES ||--o{ USERS : asigna
    USERS ||--o| PROFILES : tiene
    USERS ||--o| MEMBERS : representa
    MEMBERS ||--o{ MEMBERSHIPS : contrata
    PLANS ||--o{ MEMBERSHIPS : define
    MEMBERS ||--o{ PAYMENTS : realiza
    MEMBERS ||--o{ ATTENDANCE : registra
    PRODUCTS ||--o{ INVENTORY_MOVEMENTS : registra
    USERS ||--o{ TICKETS : crea
    USERS ||--o{ AUDIT_LOGS : genera
```

Reglas centrales:

- Las contraseñas se almacenan mediante `bcrypt`; nunca en texto claro.
- Las consultas usan parámetros para evitar inyección SQL.
- JWT controla la sesión y los guards protegen las rutas privadas.
- RBAC impide que un usuario común acceda a la administración.
- La edad se calcula con día, mes y año; menores de 14 años son rechazados.
- Entre 14 y 17 años se solicita acudiente y consentimiento.
- El servidor calcula subtotales y totales del pedido.
- Los precios no confirmados impiden completar un pedido.
- No se almacenan números completos de tarjeta ni códigos CVV.
- Las operaciones sensibles quedan registradas en auditoría.

9. Validación y cierre

La entrega se considera completa porque se verificaron los siguientes resultados:

1. El frontend compila en modo producción con Angular 22.
2. Las rutas públicas y privadas están declaradas y protegidas según su función.
3. La suite de pruebas cubre componentes, API, base de datos, consentimiento, carrito, pedidos y roles.
4. El diseño mantiene navegación, contraste, foco, textos alternativos y comportamiento responsive.
5. La documentación permite ejecutar, administrar y actualizar el proyecto.
6. El flujo de visitante, autenticación, compra informativa/operativa y soporte está representado en wireframe y diagrama.

Comandos de validación:

```
npm test -- --watch=false
npm run build
npm run db:test
npm run api:test
npm run phase3:test
npm run cart:test
npm run rbac:test
git diff --check
```

10. Conclusión

Zona Fitness entrega una solución web coherente con la identidad del gimnasio, organizada por capas y preparada para crecer. La documentación, el DOM, el wireframe y el diagrama de flujo describen el mismo producto y permiten justificar cada decisión de diseño y desarrollo.

Con la implementación, las pruebas y los entregables documentales completados, se declara que **todas las metas definidas para el proyecto fueron alcanzadas**. Las futuras ampliaciones —como nuevas pasarelas de pago, automatizaciones de marketing o integraciones externas— corresponden a una nueva fase y no constituyen pendientes del alcance final presentado.

11. Referencias del repositorio

- README del proyecto
- Guía técnica de Zona Fitness
- Arquitectura de base de datos
- Diagrama de flujo existente
- Entregables y sitemap
- Manual del cliente
- Propuesta técnica